贪心搬箱：每一步怎么定「搬哪个箱、搬到哪」

上港堆场整理 — `greedy` 包

1　一句话

算法**反复做同一件事**：在当前堆场上，挑**一个**合法搬箱动作——从某堆**顶**上取走一箱，放到**另一堆顶**上——执行后进入下一步，直到要么全场排好，要么再也挑不出合法动作。

关键：每一步**只动顶箱**，且**不能同堆内倒层**（只能从堆 i 搬到堆 k，$i \neq k$）。

2　每一步的六步流程（白话）

下面用「当前已经搬了 m 步」时的状态说明。记全场逆序箱数为 N，初始为 $N^{(0)}$，允许的上限为

$$G \;=\lfloor 1.5 \times N^{(0)} \rfloor, \qquad G = m + N \;\text{（搬箱步数 + 当前逆序箱数）}.$$

第 1 步　先看哪些堆「值得动」

只从下面两类堆里往外搬箱（源堆）：

- 这堆还有逆序（$N_i > 0$），或
- 这堆还没排好（自下而上关序号不是「下面大、上面小」）。

其它已经排好的空问题堆，这一步**不会**从它往外搬。

第 2 步　定「搬哪个箱」——只能是源堆最顶上那一箱

对每个源堆 i，没有选择权：**被搬的箱已经定了**，就是堆 i 最上面那一箱（顶箱）。程序里记它的秩为 n，源层号 $j = |S_i|$（当前堆高）。

第 3 步　定「能搬到哪」——枚举所有别的堆的顶

对源堆 i，逐个看目标堆 k（$k \neq i$）：

- 若堆 k 已满（层数 $= H$），跳过；
- 否则允许：把该顶箱放到堆 k 的**新顶层**（层号 $\ell = |S_k| + 1$）。

于是得到一个候选：「**堆 i 的顶箱 → 堆 k 的顶**」。所有源堆 × 所有能接箱的目标堆，组成这一步的**候选动作表**。

第 4 步　对每个候选先「试搬」再打分，不合格的划掉

对表里每一条「i 顶箱 → k 顶」，在内存里复制当前布局、模拟搬一次（不真改现场），然后检查：

(a) 搬完之后 $G' = (m+1) + N'$ 是否 $\le G$ ？超过则**淘汰**（翻箱代价超标）；

(b) 搬完之后全场逆序 N' 是否比搬之前 N **更大**？若更大则**淘汰**（这一步不许让逆序变多）；

(c) 搬完后的整盘布局是否**以前出现过**？若重复则**淘汰**（防止来回打转）。

剩下的就是**合法候选**。

第 5 步 在合法候选里挑「最好」的一条——这就是本步真正执行的动作

对每条合法候选算三个数（越大越好 / 越小越好见下）：

- $\Delta N = N - N'$：搬完逆序**少了几箱**（优先**尽量大**）；
- $G' = (m+1) + N'$：搬完的综合代价（在 ΔN 相同时优先**尽量小**）；
- 多贝时加分 B：源堆在初始逆序最多的贝（`drain_bay`）+2，目标堆在初始逆序最少的贝（`fill_bay`）+1（在仍打平时优先**大**）。

排序规则（从上到下比，先比赢的优先）：

$$\Delta N \text{ 大 } \rightarrow G' \text{ 小 } \rightarrow B \text{ 大 } \rightarrow N' \text{ 小.}$$

取排在最前面的一档；若这一档还有多条，**随机抽一条**执行。

第 6 步 真搬、记步、进入下一轮

把选中的「堆 i^* 顶箱 $\rightarrow$ 堆 k^* 顶」写入动作序列，更新现场堆场，$m \leftarrow m+1$，把新布局记入「已出现」集合。若此时每一堆都已良序，**结束**；否则回到第 1 步。

3 小例子（帮助对齐直觉）

假设当前只有两堆有问题：堆 3 顶箱关序号偏大（有逆序），堆 7 还能再接一箱。

- **搬哪个箱？**若 $3 \in \mathcal{I}_{\text{src}}$，本步只可能搬**堆 3 顶箱**，不能搬堆 3 下面的箱。
- **搬到哪？**若堆 7 未满，则产生候选「3 顶 $\rightarrow$ 7 顶」；若堆 $5, 7, 9$ 都能接，则产生三条候选，分别试搬、过滤、再比 ΔN 和 G'。
- **为何可能不搬 $3 \rightarrow 5$？**试搬后若 N 反而变大，或 G' 超过 G ，或布局重复，该候选会被划掉，最终可能只剩 $3 \rightarrow 7$ 合法。

4 指标（需要时再查公式）

4.1 什么叫「排好」

从一堆**底**往**顶**看关序号：每一层都要比上面一层**更大**（先提的箱关序号更小）。满足则该堆良序。

4.2 逆序箱数 N_i（单堆）

从第 2 层往上找**第一处**「上面关序号 $>$ 下面关序号」的层 j^*；从 j^* 到堆顶所有非空箱的个数，即该堆 N_i。全场 $N = \sum_i N_i$。

4.3 为何用 $G = m + N$

每搬一步 m 加 1，逆序 N 可能变；要求任意时刻 $m + N \le \lfloor 1.5N^{(0)} \rfloor$，控制「搬箱次数 + 剩余乱度」的总代价，而不是要求 N 必须比开局更小。

5 何时停

- **成功：**所有堆都良序；
- **卡住：**第 4 步后合法候选为空（在不能增 N、且 $G \le G$ 的前提下无路可走）；
- **保底：**步数达到程序上限 `hard_cap`。

6 和代码的对应

白话步骤	函数
哪些堆当源堆	`greedy_plan._src_stacks`
顶箱搬到哪些堆顶	`greedy_plan._enumerate_cross`
试搬 + 三条过滤	`simulate_move` + `_pick_best_move`
比 $\Delta N, G', B$ 并随机 tie	`_pick_best_move` 内排序
真搬并循环	`apply_move` + `plan_greedy_until_terminal`